

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Trabajo Final Integrador

Programación II

Integrantes

Giardini Silvia	Comisión: 07
Marina Giselle Cordero	Comisión: 07
Alex Dauria	Comisión: 06
Matías Perdigués	Comisión: 01

Profesores

Luciano Chirolí	Comisión: 07
Federico Frankerberger	Comisión: 06
Renzo Sosa	Comisión: 01

Fecha de Entrega
17 de Noviembre de 2025

Índice

Introducción	2
1. Integrantes y Roles	2
2. Elección del dominio y justificación	2
3. Diseño: decisiones clave y UML	2
4: Arquitectura por capas	3
1. Capa Config	3
2. Capa Entities	3
3. Capa DAO	3
4. Capa Service	3
5. Capa UI / Main	4
5. Persistencia: estructura de la base, orden de operaciones y transacciones	4
1. Estructura de la base de datos	4
2. Orden recomendado de operaciones	4
3. Manejo de transacciones (commit y rollback)	5
4. Uso de PreparedStatement	5
6. Validaciones y reglas de negocio	5
1. Validaciones principales	5
2. Reglas vinculadas a la relación 1→1	6
3. Baja lógica	6
4. Validaciones previas a transacciones	
7. Pruebas realizadas	6
1. Pruebas mediante el menú de consola	6
2. Pruebas en MySQL Workbench	7
3. Resultados generales de las pruebas	7
8. Conclusiones y mejoras futuras	8
9. Fuentes y herramientas utilizadas	9
1. Herramientas de desarrollo	9
2. Documentación consultada	9
3. Uso de Inteligencia Artificial	9
Link del repositorio en GitHub	[Link]
Link del video explicativo	[Link]

Introducción

El objetivo del proyecto es desarrollar una aplicación en Java que modele una relación unidireccional **1 a 1** entre las clases **Empleado** y **Legajo**, implementando persistencia en MySQL mediante JDBC, el patrón DAO y una capa Service responsable de las validaciones y transacciones (commit y rollback). El sistema sigue una arquitectura por capas que favorece la claridad, el mantenimiento y la escalabilidad.

1. Integrantes y Roles

- **Marina Giselle Cordero:** Diseño del dominio, entidades, scripts SQL, conexión JDBC, pruebas y documentación.
- **Silvia Giardini:** Implementación completa de la capa DAO (CRUD con JDBC y manejo de excepciones).
- **Alex Dauria:** Desarrollo de la capa Service, validaciones de negocio y coordinación de transacciones ACID.
- **Matías Perdigués:** Implementación del menú por consola, flujo UI y pruebas funcionales del sistema.

2. Elección del dominio y justificación

El dominio **Empleado → Legajo (1→1 unidireccional)** fue elegido por su claridad conceptual y su uso frecuente en áreas de Recursos Humanos. Permite aplicar de forma directa los contenidos de Programación II: modelado de entidades, persistencia con JDBC, validaciones de negocio y manejo de transacciones. Este dominio facilita integrar teoría y práctica sin agregar complejidad innecesaria, manteniendo un caso realista, escalable y coherente con una arquitectura por capas.

3. Diseño: decisiones clave y UML [Ver carpeta en GitHub](#)

Se definió una relación **1 a 1 unidireccional**, donde el empleado posee un único legajo. En la base de datos se implementó mediante una **FK con restricción UNIQUE**, en vez de clave primaria compartida, por ser más simple de mantener y más adecuada para operaciones CRUD con JDBC.

El UML del proyecto modela:

- Herencia desde BaseEntity.
- Asociación 1→1 Empleado–Legajo.
- Uso de Estado (enum).
- Interacción entre DAO, Service y capas superiores.

4. Arquitectura por capas

El proyecto implementa una **arquitectura por capas** que organiza el código según responsabilidades claramente definidas, lo que mejora la **claridad**, el **mantenimiento**, la **modularidad** y la **escalabilidad** del sistema. Cada paquete cumple funciones específicas y se comunica únicamente con las capas que corresponden, garantizando un flujo ordenado y evitando dependencias innecesarias entre componentes.

4.1. Capa Config [Ver carpeta en GitHub](#)

Incluye:

- **DatabaseConnection**: administra la conexión a MySQL utilizando un archivo de propiedades externo.
- **TransactionManager**: coordina operaciones ACID (commit y rollback) en las acciones que afectan más de una tabla.

Su responsabilidad es **centralizar la configuración**, evitar duplicación de código y garantizar conexiones seguras.

4.2. Capa Entities [Ver carpeta en GitHub](#)

Modela el **dominio** del sistema:

- Empleado
- Legajo
- BaseEntity
- Estado (enum)

Incluye atributos, constructores y getters/setters. Define la estructura lógica que luego se persistirá en la base de datos.

4.3. Capa DAO (Data Access Object) [Ver carpeta en GitHub](#)

Responsable de la **persistencia** mediante JDBC:

- **Interfaces**: GenericDAO, EmpleadoDAO, LegajoDAO
- **Implementaciones JDBC**: EmpleadoDAOImpl, LegajoDAOImpl

Se encargan de:

- CRUD completo
- Uso obligatorio de PreparedStatement
- Manejo explícito de conexiones, sentencias y ResultSets
- Operaciones autónomas y transaccionales

4.4. Capa Service [Ver carpeta en GitHub](#)

Contiene la **lógica de negocio**, validaciones y reglas del sistema:

- EmpleadoService / EmpleadoServiceImpl
- LegajoService / LegajoServiceImpl

Responsabilidades:

- Validar datos antes de persistir
- Aplicar reglas del dominio (1→1 unidireccional)
- Coordinar transacciones: al crear empleados con legajo, actualizar o realizar bajas lógicas
- Interactuar con DAO sin exponer detalles de la base de datos a la UI

4.5. Capa UI / Main [Ver carpeta en GitHub](#)

Incluye el menú por consola:

- Recibe datos del usuario
- Invoca servicios
- Muestra resultados y mensajes
- Controla el flujo de ejecución del programa

Permite realizar todas las operaciones CRUD del proyecto.

5. Persistencia: estructura de la base, orden de operaciones y transacciones

La persistencia del sistema se implementó utilizando **MySQL** y el conector **JDBC**, con dos tablas principales: empleados y legajos. La relación se establece mediante una clave foránea única, garantizando la correspondencia 1→1.

5.1. Estructura de la base de datos

Tabla empleados

- id (PK, autoincremental)
- eliminado (BOOLEAN, baja lógica)
- nombre, apellido, dni, email, area
- fecha_ingreso

Incluye restricciones importantes como **DNI único** y validación de formato de email.

Tabla legajos

- id (PK, autoincremental)
- eliminado (BOOLEAN)
- nro_legajo (UNIQUE)
- categoria, estado, fecha_alta, observaciones
- empleado_id (FK UNIQUE → empleados.id)

5.2. Orden recomendado de operaciones

Debido a la dependencia 1→1, las operaciones disciplinan el flujo de trabajo para evitar inconsistencias:

Creación

1. Insertar el registro en empleados.
2. Obtener el ID generado.
3. Insertar el legajo asociado usando ese ID como FK.

Actualización

- Actualizar primero el empleado y luego el legajo, manteniendo coherencia de negocio.

Baja

- Se aplica **baja lógica**, actualizando eliminado = TRUE en ambas tablas.
- Se evita borrar datos para preservar trazabilidad.

5.3. Manejo de transacciones (commit y rollback)

El sistema utiliza un **TransactionManager** propio para coordinar operaciones ACID en persistencia.

Transacciones con commit

Se utiliza commit() cuando:

- Se completa exitosamente la creación conjunta de Empleado–Legajo
- Se actualizan ambos registros sin errores
- Se realiza una baja lógica de forma coherente

Esto asegura que **ambas tablas se mantengan sincronizadas**.

Transacciones con rollback

Se ejecuta rollback() cuando ocurre:

- Error al insertar legajo después del empleado
- Fallo en validaciones previas a persistir
- Excepciones de conexión o prepared statements

El rollback garantiza que **ningún registro quede a medio crear o modificar**, manteniendo la integridad de datos.

5.4. Uso de PreparedStatement

Todas las operaciones CRUD se implementan con PreparedStatement, lo cual:

- Previene inyección SQL
- Mejora el rendimiento
- Evita errores por concatenación de cadenas
- Facilita el paso seguro de parámetros

5.5. Integridad y consistencia

Las decisiones de persistencia aseguran:

- Relación 1→1 estricta
- Eliminaciones limpias (baja lógica coherente)
- Transacciones seguras
- Validaciones aplicadas antes de interactuar con la base
- Evitar registros huérfanos o duplicados

6. Validaciones y reglas de negocio

La capa Service implementa las **validaciones de negocio** necesarias para garantizar que los datos sean coherentes antes de persistirlos en la base de datos. Estas reglas se aplican previo a la llamada a los DAO y forman parte de la lógica central del sistema.

6.1. Validaciones principales

✓ DNI único

Antes de crear un empleado, el sistema verifica que no exista otro registro con el mismo DNI. Evita duplicidad y mantiene integridad lógica.

✓ Email válido

Se valida el formato para asegurar un correo estructuralmente correcto (usuario@dominio.ext). Evita registros inconsistentes y valores sin sentido.

✓ Fechas válidas

- fechaIngreso y fechaAlta **no pueden ser futuras**.
- Se verifica que no sean nulas.
- Estas condiciones aseguran que los datos respondan a un escenario realista de RRHH.

✓ Campos obligatorios

Nombre, apellido, DNI, número de legajo y área no pueden estar vacíos.

Evita inserciones incompletas.

6.2. Reglas vinculadas a la relación 1→1

✓ Un empleado solo puede tener un legajo

El Service verifica que no exista un legajo asociado previamente al mismo empleado.

✓ Un legajo no puede existir sin un empleado

La operación de creación conjunta se hace en una **única transacción**, garantizando integridad.

✓ Actualización coherente

Se actualizan los datos del empleado y su legajo manteniendo la transacción como una sola unidad de trabajo.

6.3. Baja lógica

El sistema no elimina físicamente registros.

En su lugar, aplica **baja lógica** mediante el atributo eliminado = TRUE:

- Preserva el historial
- Evita pérdida de información
- Evita inconsistencias si se reactivan datos

6.4. Validaciones previas a transacciones

La capa Service verifica todas las reglas antes de ejecutar operaciones sobre la base:

- Si alguna validación falla → se cancela la operación
- No se abre transacción innecesariamente
- No se llega a escribir datos erróneos

Esto reduce errores y asegura la consistencia global del sistema.

7. Pruebas realizadas (consola y consultas SQL)

Las pruebas del sistema se realizaron desde dos frentes:

1. **Ejecutando el menú por consola (UI)**
2. **Verificando los resultados en la base de datos con consultas SQL**

El objetivo fue comprobar el correcto funcionamiento de las operaciones CRUD, las validaciones y las transacciones ACID.

7.1. Pruebas mediante el menú de consola

A partir de la clase AppMenu, se probaron las siguientes funcionalidades:

✓ Creación de Empleado + Legajo (transacción conjunta)

- Ingreso de datos por consola.
- Confirmación en pantalla del alta.
- Verificación en MySQL de que ambos registros fueron creados en una única transacción.

✓ Listado de empleados

- Se mostraron solo los registros con eliminado = FALSE.
- Se verificó que el legajo asociado se muestra correctamente.

✓ Búsqueda por DNI

- Prueba con DNI existente (retorna empleado + legajo).
- Prueba con DNI inexistente (mensaje de error).
- Comprobación de validación de formato.

✓ Actualización de datos

- Modificación de nombre, email, categoría o estado del legajo.
- Confirmación en consola y base de datos.
- Comprobación de transacción en caso de errores.

✓ Baja lógica

- Se marcó eliminado = TRUE en empleado y legajo.
- Verificación de que ya no aparece en listados.
- Se confirma que no se elimina físicamente.

7.2. Pruebas en MySQL Workbench

Después de cada operación se ejecutaron consultas SQL para comprobar la integridad:

Consulta general de empleados

```
SELECT * FROM empleados WHERE eliminado = FALSE;
```

Consulta del legajo asociado

```
SELECT * FROM legajos WHERE empleado_id = 1;
```

Verificación de la relación 1→1

```
SELECT e.id, e.nombre, l.nro_legajo  
FROM empleados e  
JOIN legajos l ON e.id = l.empleado_id;
```

Comprobación de baja lógica

```
SELECT id, nombre, eliminado  
FROM empleados  
WHERE id = X;
```

Prueba de transacción fallida

Se simuló un error al insertar el legajo:

- Insert del empleado → OK
- Insert del legajo → Error provocado

Resultado esperado (comprobado):

```
SELECT * FROM empleados WHERE dni = '99999999';
```

→ No existe, porque se ejecutó **rollback()**.

7.3. Resultados generales de las pruebas

- Todas las operaciones CRUD funcionaron correctamente.
- Las validaciones evitaron datos inválidos.
- Las transacciones garantizaron coherencia entre ambas tablas.
- No se generaron registros huérfanos ni duplicados.
- El sistema respondió adecuadamente ante errores simulados.

8. Conclusiones y mejoras futuras

El desarrollo de este Trabajo Final Integrador permitió aplicar de forma práctica los conceptos centrales de Programación II: modelado orientado a objetos, arquitectura por capas, persistencia con JDBC, validaciones de negocio y manejo de transacciones ACID. La relación **1 a 1 unidireccional** entre Empleado y Legajo se implementó correctamente tanto en el modelo Java como en la base de datos, garantizando integridad referencial y consistencia de los registros.

La arquitectura por capas facilitó la separación de responsabilidades y permitió un sistema más ordenado, mantenible y escalable. La capa DAO resolvió la interacción con MySQL de manera segura mediante PreparedStatement, mientras que la capa Service centralizó las reglas de negocio y las operaciones transaccionales, evitando inconsistencias y registros incompletos. Las pruebas realizadas desde la consola y las consultas SQL confirmaron el correcto funcionamiento del CRUD, de la baja lógica y de las validaciones aplicadas.

En conjunto, el proyecto demuestra una implementación sólida, coherente y ajustada a las consignas académicas, integrando teoría y práctica en un caso de uso realista.

Se identifican posibles extensiones para seguir evolucionando el sistema:

- **Agregar interfaz gráfica (JavaFX)** para una experiencia de usuario más intuitiva.
- **Incorporar logs** de auditoría mediante una librería como Log4j o java.util.logging.
- **Implementar pruebas unitarias** con JUnit para validar métodos críticos.
- **Crear una API REST** utilizando frameworks como Spring Boot para facilitar acceso remoto.
- **Agregar nuevas relaciones** (por ejemplo, Empleado → Contratos o Empleado → Asistencias).
- **Migrar a un ORM (Hibernate/JPA)** para reducir la complejidad del código JDBC.

Estas mejoras permitirían potenciar el sistema, ampliar funcionalidades y profesionalizar aún más la solución.

9. Fuentes y herramientas utilizadas

Para el desarrollo del proyecto se utilizaron herramientas, recursos y documentación técnica que permitieron asegurar una correcta implementación del modelo, la arquitectura y las operaciones de persistencia.

9.1. Herramientas de desarrollo

- **Java JDK 21** – Lenguaje principal del proyecto.
- **NetBeans 21** – IDE utilizado para codificación, pruebas y ejecución.
- **MySQL Server 8.4.x** – Gestor de base de datos.
- **MySQL Workbench** – Herramienta para consultas SQL y verificación de registros.
- **MySQL Connector/J 8.4.0** – Conector JDBC para comunicación con la base.
- **GitHub** – Control de versiones y repositorio del proyecto.
- **UMLtino / Diagramas UML** – Creación del documento visual de arquitectura y entidades.

9.2. Documentación consultada

- Oracle Java Documentation: <https://docs.oracle.com/en/java>
- JDBC API Specification
- MySQL Documentation: <https://dev.mysql.com/doc/>
- Tutoriales oficiales de NetBeans y JDBC
- Material teórico brindado por la cátedra de Programación II (UTN).

9.3. Uso de Inteligencia Artificial

Se utilizó IA como asistente técnico para:

- Revisar estructura del proyecto
- Redactar documentación
- Verificar lógica de transacciones y persistencia
- Organizar secciones del informe
- Proveer explicaciones y ejemplos complementarios

Toda la codificación, diseño UML, scripts SQL y pruebas fueron realizadas por los integrantes del equipo.